

# The ROS 2 cheat sheet

The concepts, the commands you type every day, one complete node, and how to move a robot. Page 2 has the six-project roadmap, the tools, a debugging checklist and every link from the article.

Works for Humble, Jazzy and Lyrical. Written by Bharat Jain, October 2026, version 1.0.

## The ideas

|           |                                                                                                                    |
|-----------|--------------------------------------------------------------------------------------------------------------------|
| Node      | One program with one job: camera driver, planner, motor controller.<br>Python: rclpy. C++: rclcpp.                 |
| Topic     | A named stream. Publishers write, any number of subscribers read, nobody replies: /scan, /cmd_vel.                 |
| Service   | One request, one reply, like “spawn a turtle at (2, 2)”.                                                           |
| Action    | A long task you can watch and cancel: “navigate to this pose”.                                                     |
| Parameter | A node’s settings, changeable while it runs: max_speed, use_sim_time.                                              |
| Interface | The typed data on the wire: .msg, .srv, .action files. Common ones: table on the right.                            |
| Package   | A folder of nodes with a package.xml. A workspace holds many packages under src/.                                  |
| Launch    | A .launch.py file starts several nodes with their parameters in one go.                                            |
| tf2       | The tree of coordinate frames, map → odom → base_link → laser. Every reading lives in a frame.                     |
| QoS       | Delivery settings per topic. Sensors publish <i>best effort</i> ; a <i>reliable</i> subscriber then hears nothing. |

## Workspace build once, source everywhere

```
ros2_ws/src/my_pkg/  
package.xml      name, dependencies  
setup.py         entry_points → ros2 run  
my_pkg/bumper.py your nodes  
  
source /opt/ros/jazzy/setup.bash  
cd ~/ros2_ws/src  
ros2 pkg create --build-type ament_python \  
  my_pkg  
cd ~/ros2_ws && colcon build --symlink-install  
source install/setup.bash  
  
Build from the workspace root, not inside src. Put  
both source lines in ~/.bashrc.
```

## Commands you will type every day

```
ros2 run pkg executable  
Start one node  
  
ros2 launch pkg robot.launch.py use_sim_time:=  
rue  
Start several nodes, with arguments  
  
ros2 node list  
ros2 node info /node  
What is running; what a node publishes, subscribes to  
and serves  
  
ros2 topic list -t  
ros2 topic info -v /topic  
ros2 interface show geometry_msgs/msg/Twist  
Topics with types; who publishes and subscribes, with  
QoS; the fields of a type  
  
ros2 topic echo /scan --once  
ros2 topic hz /scan  
Print one message; measure the rate  
  
ros2 topic pub -r 10 /cmd_vel geometry_msgs/  
msg/Twist "{linear: {x: 0.2}, angular: {z: 0.5}}"  
Publish from the terminal, ten times a second  
  
ros2 service list -t  
ros2 service call /spawn turtlesim/srv/Spawn "  
{x: 2.0, y: 2.0, name: 't2'}"  
List services; call one with a YAML request
```

```
ros2 action list -t  
ros2 action send_goal /navigate_to_pose nav2_ms  
gs/action/NavigateToPose "{pose: {header: {fram  
e_id: map}, pose: {position: {x: 1.0}, orientat  
ion: {w: 1.0}}}}}" --feedback  
List actions; send a goal and watch the feedback  
  
ros2 param list  
ros2 param get /node name  
ros2 param set /node name value  
Read and change settings while the node runs
```

```
ros2 bag record -a -o run1  
ros2 bag play run1  
Record every topic; replay it later without the robot  
  
rviz2  
rqt_graph  
ros2 doctor --report  
Sensors and map in 3D; the live node graph; check the  
install
```

## One complete node subscribe, decide, publish

```
import rclpy  
from rclpy.node import Node  
from geometry_msgs.msg import Twist  
from sensor_msgs.msg import LaserScan  
  
class Bumper(Node):  
    def __init__(self):  
        super().__init__('bumper')  
        self.pub = self.create_publisher(Twist, '/cmd_vel', 10)  
        self.create_subscription(LaserScan, '/scan',  
                                self.on_scan, 10)  
        self.create_timer(0.1, self.tick) # 10 Hz  
        self.ahead = 9.9  
  
    def on_scan(self, msg):  
        self.ahead = min(r for r in msg.ranges if r > 0.05)  
  
    def tick(self):  
        cmd = Twist()  
        if self.ahead > 0.5:  
            cmd.linear.x = 0.2 # m/s, forward  
        else:  
            cmd.angular.z = 0.6 # rad/s, turn left  
        self.pub.publish(cmd)  
  
def main():  
    rclpy.init()  
    rclpy.spin(Bumper())  
    rclpy.shutdown()
```

**What it does.** Drives forward until the LiDAR sees something within half a metre, then turns left until the way is clear.

**The three parts** of almost every node: a publisher that sends Twist commands, a subscriber whose callback runs on each LaserScan, and a timer that decides ten times a second. spin() keeps them running.

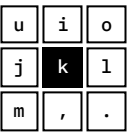
**Run it.** Save as my\_pkg/bumper.py, add 'bumper = my\_pkg.bumper:main' under console\_scripts in setup.py, build, then ros2 run my\_pkg bumper.

## Moving a robot the /cmd\_vel topic

linear.x forward speed in m/s; 0.2 is a safe start

angular.z turning speed in rad/s, positive turns left; try 0.5

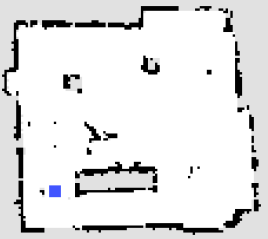
ros2 run teleop\_twist\_keyboard  
teleop\_twist\_keyboard drives anything that listens on /cmd\_vel:



i forward, , back, j and l turn on the spot, corners drive in arcs, k or any other key stops. q/z change both speeds by 10%, w/x linear only, e/c angular only.

A real map: my room, built by a \$200 LiDAR rover in Project 2. White is free, black is occupied, grey is unknown. The blue square is the robot.

1 m = 20 cells of 5 cm



## Messages you will meet

|                                    |                                                              |
|------------------------------------|--------------------------------------------------------------|
| Twist<br>geometry_msgs/msg         | A velocity command: linear.x and angular.z                   |
| LaserScan<br>sensor_msgs/msg       | One LiDAR sweep: ranges[] plus the angles they were taken at |
| Image<br>sensor_msgs/msg           | A camera frame; cv_bridge turns it into an OpenCV image      |
| Odometry<br>nav_msgs/msg           | Where the wheels think the robot is, and how fast it moves   |
| OccupancyGrid<br>nav_msgs/msg      | The map: cells of -1 unknown, 0 free, 100 occupied           |
| PoseStamped<br>geometry_msgs/msg   | A position and heading in a named frame, such as a goal      |
| NavigateToPose<br>nav2_msgs/action | The action behind “drive there”                              |

|                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>0</div> <div>Turtlesim</div> <div>The hello world. One node draws a turtle, another turns your arrow keys into velocity messages. Watch the topic to see them talk.</div> <div>ros2 run turtlesim turtlesim_node, turtle_teleop_key, ros2 topic echo /turtle1/cmd_vel</div> | <div>1</div> <div>Teleop a real robot</div> <div>Drive a wheeled robot from your laptop over /cmd_vel. Run nodes across several terminals and watch the sensors in RViz.</div> <div>TurtleBot3 or Waveshare UGV, teleop_twist_keyboard, rviz2</div> | <div>2</div> <div>Map a room</div> <div>SLAM builds an occupancy grid while you drive. Save it, give Nav2 a goal in RViz, and the robot plans a path and drives around whatever is in the way.</div> <div>slam_toolbox, map_saver_cli, nav2_bringup</div> | <div>3</div> <div>Frontier exploration</div> <div>Your own node. Frontiers are free cells next to unknown ones: cluster them, score by size and distance, send the best to Nav2, repeat until none remain.</div> <div>OccupancyGrid, NavigateToPose, a coding agent</div> | <div>4</div> <div>Semantic navigation</div> <div>Capture images while exploring, let a vision-language model label them, store labels with poses in a vector database. “Go to the couch” finds the nearest match.</div> <div>a VLM, Chroma, Nav2</div> | <div>5</div> <div>Talk to the robot</div> <div>The ROS-MCP server exposes topics, services and actions as tools. Claude Code or Codex reads the sensors, plans, and drives from plain English.</div> <div>ros-mcp-server, rosbridge_server</div> | <div>6</div> <div>A robot arm</div> <div>Build the \$200 SO-101, wrap it in ROS 2, let MoveIt do the inverse kinematics. Record trajectories with the leader arm, replay them by hand or by LLM.</div> <div>LeRobot, ros2_control, MoveIt 2</div> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Tools, one line each

RViz2

3D view of frames, LiDAR, camera, map and paths. Set the fixed frame to map first.

rqt\_graph

Live picture of nodes and topics; the fastest way to spot a missing link.

Gazebo

Physics simulator, `gz sim`; `ros_gz_bridge` carries topics across.

SLAM Toolbox

Builds 2D maps from LiDAR and odometry. Start with `online_async_launch.py`.

Nav2

Costmaps, planners, controllers and recoveries behind one action, `NavigateToPose`.

MoveIt 2

Inverse kinematics, collision-free planning and execution for arms.

tf2

`ros2 run tf2_tools view_frames` draws the frame tree; `tf2_echo map base_link` prints one transform.

ros2 bag

Record one drive, debug against it a hundred times.

When nothing works

1. Did this terminal source the workspace? `source install/setup.bash`, then `ros2 pkg list | grep my_pkg`.

2. Is the node running, and is the topic name exactly right? `ros2 node list`, `ros2 topic list`. Mind slashes and namespaces.

3. `echo` shows nothing although a publisher exists: QoS. Check `ros2 topic info -v`, then `ros2 topic echo --qos-reliability best_effort /scan`.

4. Two machines cannot see each other: same `ROS_DOMAIN_ID`, same network, multicast allowed. Test with `ros2 multicast send / receive`.

5. The robot ignores you: `ros2 topic echo /cmd_vel`. If commands arrive, check the motor enable, the e-stop and the robot's real topic name.

6. In simulation every node needs `use_sim_time:=true`, or `tf` and `Nav2` complain about time.

7. Build fails: `colcon build --packages-select my_pkg --event-handlers console_direct+` shows the real error. Missing dependency: `rosdep install --from-paths src --ignore-src -y`.

8. Still stuck: `ros2 doctor --report`, then search the exact error on Robotics Stack Exchange.

Which ROS 2 to install

| Distribution                    | Ubuntu | Until    |
|---------------------------------|--------|----------|
| Lyrical Luth, LTS, May 2026     | 26.04  | May 2031 |
| Jazzy Jalisco, LTS, May 2024    | 24.04  | May 2029 |
| Humble Hawksbill, LTS, May 2022 | 22.04  | May 2027 |

Jazzy has the most tutorials and robot packages today; Lyrical is the newest. ROS 1 ended in May 2025. Install the `-desktop` variant so `RViz` comes along.

Hardware that works

A Raspberry Pi 5 or Jetson Orin Nano Super as the brain, a Slamtec RPLIDAR C1 for mapping, a TurtleBot3 Burger or Waveshare UGV Rover (both ship with ROS 2 packages), the SO-101 arm for manipulation. Ubuntu 24.04 on x86 is easiest; on a Mac, Ubuntu inside UTM (`mac.getutm.app`) runs all but heavy Gazebo scenes.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>Links every address from the article, typed once</div> <div><div>Learn</div><div><div>ROS 2 documentation, with the install guide</div><div>docs.ros.org/en/jazzy</div></div><div><div>Tutorials, from the CLI to your first package</div><div>docs.ros.org/en/jazzy/Tutorials.html</div></div><div><div>Turtlesim guide</div><div>docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Introducing-Turtlesim/Introducing-Turtlesim.html</div></div><div><div>Build a robot from scratch, Articulated Robotics</div><div>articulatedrobotics.xyz/tutorials/mobile-robot/project-overview</div></div><div><div>Ask questions</div><div>robotics.stackexchange.com</div><div>discourse.openrobotics.org</div></div></div> | <div>Tools</div> <div><div>Nav2</div><div>docs.nav2.org</div></div> <div><div>SLAM Toolbox</div><div>github.com/SteveMacenski/slam_toolbox</div></div> <div><div>MoveIt 2</div><div>moveit.picknik.ai</div></div> <div><div>Gazebo with ROS 2</div><div>gazebosim.org/docs/harmonic/ros_installation</div></div> <div><div>teleop_twist_keyboard</div><div>github.com/ros2/teleop_twist_keyboard</div></div> | <div>Projects 3 to 5</div> <div><div>Frontier exploration, a reference implementation</div><div>github.com/robo-friends/m-explore-ros2</div></div> <div><div>The original frontier paper, Yamauchi 1997</div><div>ieeexplore.ieee.org/document/613851</div></div> <div><div>Semantic maps, VLMaps</div><div>github.com/vlmaps/vlmaps</div></div> <div><div>ROS-MCP server</div><div>github.com/robotmcp/ros-mcp-server</div></div> <div><div>Model Context Protocol</div><div>modelcontextprotocol.io</div></div> | <div>Robots and parts</div> <div><div>TurtleBot3: simulation, SLAM, navigation</div><div>emmanual.robotis.com/docs/en/platform/turtlebot3/overview</div></div> <div><div>Waveshare UGV Rover, ROS 2 kit</div><div>waveshare.com/ugv-rover-ros2-kit.htm</div></div> <div><div>RPLIDAR C1 and its ROS 2 driver</div><div>slamtec.com/en/c1</div><div>github.com/Slamtec/sllidar_ros2</div></div> <div><div>SO-101 arm: guide, parts, kits</div><div>huggingface.co/docs/leobot/so101</div><div>github.com/TheRobotStudio/SO-ARM100</div></div> <div><div>My code, videos and the article</div><div>github.com/itsbharatj</div><div>x.com/BharatJain8873</div><div>YOUTUBE_URL</div></div> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|